Coverage-Preserving Compilation v2: A 31-Source Pipeline with Joint Manual and Knowledge-Graph Production

Pedro Farinha Independent Researcher pedro.farinha@shiftleft.pt

Supported by Shiftleft — Secure Software Engineering, lda.

Abstract

Background. The Security-by-Design Theory of Everything (SbD-ToE) research programme has produced two upstream artefacts: a normalized application-security ontology (AppSec Core v1; 10 slices, 259 typed instances; [4]) and an iterative coverage-preserving compilation method that operationalises a two-stage normalization pipeline grounding heterogeneous normative sources against that ontology under explicit bounding conditions ([5]). What remains open at the close of the programme's first multi-cycle arc is the *downstream* reconciliation primitive. When new sources enter the normalized substrate, the SbD-ToE Manual — the practitioner-authored consolidation of security-by-design engineering experience that is the programme's compiled knowledge base — and its derived knowledge graph must be kept in sync with the substrate: each entity surfaced by the substrate is reconciled against the Manual, either by demonstrating that existing prose already covers the entity (and updating the traceability surface) or by surfacing additions needed in chapter re-prose or new sections. The same reconciliation operation serves a second recurring use case: the Manual is open source (registered IGAC 949/2025; CC BY-SA 4.0) and may be customized by end users for their own deployments, in which case the customized Manual's knowledge graph — consumed by the user's MCP instance — must be recompiled against the customization. Both use cases require an operational pipeline that takes a normalized substrate (or a customized Manual) as input and produces a coherent published Manual + knowledge graph pair as output, exposing gaps explicitly enough that subsequent re-executions can proceed against new inputs without rediscovering the structure each time.

Objective. This paper publishes that pipeline as a programme primitive. The pipeline composes seven stages — external-source ingest, ontology-grounded normalization, normalized-substrate emission, V1 ontology binding, Manual coverage analysis, Manual content surfaces and KG compilation, and joint closure pinning with public-deposit mirroring (§3.1) — into a single repeatable artefact. The paper demonstrates the pipeline at the programme's current scale — 31 external sources, the v1 ontology, and a practitioner manual covering ten security-by-design domain families — and reports the joint frozen state that the demonstration produced.

Method. The pipeline is composed from prior programme artefacts (the ontology of [4], the design-science method of [5]) and exercised through one cycle (executed during 2026-05-09 to 2026-05-12; pinned at the cycle's closure ledger anchor `cycle-b-frozen-2026-05-12`). Six iterations are documented as one instantiation of the pipeline against the current substrate; each iteration's output is traced to a specific pipeline stage. Three closure mechanisms are quantified: entity-level traceability exposure, cross-chapter cross-references, and content-authoring registration. A three-way routing taxonomy (Core-mapped / Manual-only / Out-of-AppSec) is reported per chapter and contrasted against the programme's earlier five-source execution.

Contributions. (1) An operational pipeline composing five prior programme artefacts into a single repeatable primitive. (2) A cycle-close Manual + knowledge-graph joint artefact deposited at the cycle's closure state (closure ledger anchor `cycle-b-frozen-2026-05-12`; public deposits at `appsec-core-`

ontology-research/papers/08-pipeline-primitive-demonstration/artifacts/ and the public Manual repository SbD-ToE/sbd-toe-manual), consumable by downstream programme work — including the dual-mode MCP instrument of [10] and, by extension, the pre-registered empirical evaluation of [9]. (3) Empirical demonstration of pipeline closure at V1 scale, with 38 of 38 detected gaps resolved through three closure mechanisms (37 via traceability mechanisms with no new Manual prose; 1 registered for the future-work surface). (4) A three-way routing taxonomy (Core-mapped / Manual-only / Out-of-AppSec) reported per chapter. (5) An empirical contrast against the five-source execution of [2] documenting gap-class redistribution between executions — Semantic (claim-gap) share grows as sources diversify, while Gap (content-gap) share declines as Manual prose stabilizes.

Frozen state. The pipeline demonstration's cycle-closure state is pinned at the closure ledger anchor cycle-b-frozen-2026-05-12, an internal-governance snapshot recording closure commits across the four artefact-class origin repositories. The four artefact classes are deposited at two public surfaces: three of the four (knowledge graph runtime v1.2, substrate-grounding evidence + gap-analysis outputs, closure brief) at the public research repository appsec-core-ontology-research/papers/08-pipeline-primitive-demonstration/artifacts/ because their origin repositories are not publicly accessible, and the fourth (Manual prose corpus) at the public Manual repository SbD-ToE/sbd-toe-manual at the closure tag. A cross-cutting figshare bundle DOI is assigned at submission. All artefacts are SHA-256-pinned within the deposit chain.

Keywords: operational pipeline; ontology-grounded knowledge integration; practitioner manual; knowledge graph compilation; programme primitive; SbD-ToE; AppSec Core; cycle closure

---

1. Introduction

1.1 Programme arc

The Security-by-Design Theory of Everything (SbD-ToE) research programme [3] has produced, over a sequence of prior papers, the upstream components required to integrate heterogeneous normative security-engineering sources into auditable artefacts. Earlier programme outputs established two such components: the normalization vocabulary (AppSec Core, originally introduced in [1] and formalized at v1 in [4]) and the design-science method that grounds the vocabulary against heterogeneous sources iteratively ([2] at v0 first-wave scale; [5] at the cycle-close 31-source scale at which this paper operates). The output of those upstream components is a normalized substrate of source claims bound to V1 entities, produced by the two-stage normalization pipeline detailed in [5] and operating under the bounding conditions reported there. What remains open at the operational closure of the programme's first multi-cycle arc is the downstream reconciliation primitive: once external sources are normalized against the ontology, the SbD-ToE Manual (the practitioner-authored consolidation of security-by-design engineering experience and the programme's compiled knowledge base; registered IGAC 949/2025; CC BY-SA 4.0) and its derived knowledge graph must be kept in sync with the substrate — each entity reconciled against existing Manual prose, with closure routed to one of three paths: traceability exposure (existing prose covers the entity; surface the reference), cross-chapter routing (prose exists in a different chapter; add a pointer), or content-authoring registration (no prose addresses the entity; register the gap for a subsequent authoring round). The same reconciliation primitive operates in a second recurring use case beyond programme-internal cycle re-executions: the Manual is open source and may be customized by end users for their own deployments, in which case the customized Manual's knowledge graph — consumed by the user's MCP instance — must be recompiled against the customization. This paper publishes that downstream reconciliation as a programme primitive in its own right.

1.2 Pipeline-primitive thesis

This paper publishes that process as a programme primitive — an operational pipeline that can be re-executed in subsequent cycles against new external sources, evolved ontology versions, or expanded Manual content (the programme-internal re-execution case), and against end-user-customized Manual instances that require their derived knowledge graph recompiled for a downstream MCP deployment (the open-source customization case). Both cases produce a joint Manual + knowledge graph frozen artefact per execution. The pipeline composes seven stages (specified in §3.1): external-source ingest, ontology-grounded normalization, normalized-substrate emission, V1 ontology binding, Manual coverage analysis (with closure routed to traceability exposure, cross-chapter pointer, or content-authoring registration per §6), Manual content surfaces and knowledge-graph compilation, and joint closure pinning with public-deposit mirroring. Each stage's input, output, and decisioning artefact is bound to a stable identifier at the cycle's closure ledger anchor. The cycle reported in this paper (executed during 2026-05-09 to 2026-05-12 and pinned at the cycle's closure ledger anchor `cycle-b-frozen-2026-05-12`) is one instantiation of the pipeline against the current substrate — 31 external sources, the v1 ontology (10 slices, 259 typed instances), and a practitioner manual covering the ten domain families that the ontology partitions. The cycle reported here is the second execution of the coverage-preserving compilation method that the pipeline incorporates: the earlier execution of [2] operated at five-source first-wave scale and a smaller ontology surface, and is the reference against which §8 reports a between-execution gap-class redistribution. The pipeline is positioned for further re-execution at subsequent substrates (§10.5); whether the empirical mix observed at the present scale generalises is an empirical question for those subsequent executions.

The paper's framing is operational rather than methodological: it does not re-derive AppSec Core (whose artefact paper is [4]), the design-science cycle that grounded the ontology (whose method paper is [5]), or the manual as a standalone editorial product (registered IGAC 949/2025; CC BY-SA 4.0; cited as input). The contribution is the composition — that these prior artefacts, taken together, instantiate a pipeline that produces auditable joint snapshots, and that the pipeline at the current scale routes 38 of 38 detection-flagged entries through three classes of closure mechanism (deterministic traceability exposure, deterministic cross-chapter pointer, registered content authoring), with the empirical mix of mechanisms reported per chapter and contrasted against the programme's earlier five-source execution.

1.3 Contributions

This paper contributes five artefacts to the programme:

1. An operational pipeline composing five prior programme artefacts into a single repeatable primitive. The pipeline is specified in §3 with stage-level inputs, outputs, and decisioning surfaces, and exemplified in §4 with method-level discipline.
2. A cycle-close Manual + knowledge-graph joint artefact deposited at the cycle's closure state `cycle-b-frozen-2026-05-12` (§9) and made available at the public deposit surfaces specified in §11.1. The joint artefact is the citable deliverable consumed by downstream programme work: the knowledge-graph runtime v1.2 (§9.2) feeds the dual-mode MCP instrument specified in [10] and, by extension, the pre-registered empirical evaluation of [9]. The Manual content at the same anchor is the engineering-substance corpus that the MCP retrieval surface exposes.
3. Empirical demonstration of pipeline closure at V1 scale. The cycle's six pipeline-stage instantiations are documented in §5; 38 of 38 detected gaps are resolved through three closure mechanisms quantified in §6 (37 of 38 via traceability mechanisms with no new Manual prose; 1 of 38 registered for the future-work surface under the anti-rush content discipline).

4. A three-way routing taxonomy (Core-mapped / Manual-only / Out-of-AppSec) reported per chapter in §7, with substantive findings on chapter-level coverage asymmetry at V1 scope.
5. An empirical contrast against the 5-source baseline of [2] documenting gap-class redistribution at cycle scale-up: claim-gaps grow proportionally as sources diversify, content-gaps decline as Manual prose stabilizes (§8).

1.4 Scope of claims

Claims are bounded to operational pipeline behaviour at the V1 scale — 31 external sources, the v1 ontology, and the Manual content covered by the cycle's six iterations. The paper does not claim that the pipeline is the only valid composition of these prior artefacts, that the closure mechanisms exhaust the space of possible gap dispositions, or that the V1-scale empirical mix generalizes to substantially different substrate compositions (e.g., a Manual covering distinct domain families, or an ontology with substantially different partition cardinality). The pipeline is published as a primitive that subsequent cycles can re-execute and report against; whether the empirical mix at V1 generalizes is an empirical question for those subsequent cycles.

The Manual ontology V2 referenced in §3 (the semi-formal YAML vocabulary backing the Manual's chapter-intrinsic content) is treated as an input artefact at its frozen version; its formalization as OWL+SHACL is registered as future work in §10 and is out of scope for this paper.

---

2. Background

2.1 Programme antecedents

This paper builds on five prior programme artefacts cited throughout the body:

- AppSec Core v0 [1] — the original normalized ontology at first-wave scale (10 slices, 234 typed instances, 5 normative sources).
- The v0 coverage-preserving compilation method [2] — the earlier method paper that demonstrated coverage preservation at 5-source scale and established the gap-class taxonomy (claim-gap, cross-reference-gap, content-gap) used throughout this paper.
- AppSec Core v1 formalized artefact [4] — the v1 ontology as OWL 2 DL + SHACL apparatus, with the 259-instance populated graph published as the cycle-close artefact.
- The design-science method at cycle-close scale [5] — the 31-source iterative cycle that produced v1, exercising the ACR (AppSec Core Change Request) protocol for additive ontology evolution.
- The practitioner manual (SbD-ToE Manual, registered IGAC 949/2025, CC BY-SA 4.0) — the human-readable consolidation of security-by-design knowledge that the present paper validates and updates through the pipeline's coverage-analysis stage.

The pipeline reported here composes these artefacts. Each prior artefact remains a standalone publication; this paper does not re-derive any of them. The composition is the contribution.

2.2 Design Science Research framing

The paper operates within the Design Science Research (DSR) tradition, following the established methodological framing of Hevner et al. [6] and Wieringa [7] for design-artefact research and Peffers et al. [8] for the iterative cycle structure. The pipeline reported here is what the programme refers to as a "Type-2" DSR artefact — an instantiation of method as operational primitive — consumed by "Type-1" DSR artefacts (the Manual and KG produced per cycle). The Type-1 / Type-2 nomenclature is internal programme shorthand and does not appear in Hevner et al.'s standard taxonomy of constructs, models,

methods, and instantiations [6]; the spirit of the distinction follows Wieringa's separation between artefact and process objects of design [7]. The DSR-acceptance criterion is *good-for-intended-fit*: at the cycle's close, the pipeline produced a joint Manual + KG frozen artefact citable by downstream consumers (the pre-registered empirical evaluation of [9] and the dual-mode MCP instrument of [10]), with the closure mechanisms reported per chapter and the residual content-gap registered as future programme work.

2.3 Ontology-mediated knowledge integration

The programme's prior work [2, 5] grounded a normalized ontology against heterogeneous normative sources through a coverage-preserving compilation method. The method operates within the ontology-based information-integration tradition surveyed by Noy [13]: source heterogeneity is reconciled by grounding source items against a normalized ontology, rather than by direct source-to-source mapping. The empirical evidence at first-wave scale ([2], 5 sources) and cycle-close scale ([5], 31 sources) supports the claim that the ontology absorbs source heterogeneity as additive instance population while preserving the partition structure declared at v0. The present paper assumes this prior result and proceeds operationally: the question is no longer *whether* the ontology grounds the sources, but *how* the ontology and the grounded substrate are integrated into a published Manual + KG artefact pair that downstream programme work consumes.

2.4 Manual ontology V2

In parallel with the v1 ontology cycle, the practitioner Manual was annotated with a semi-formal YAML vocabulary — the Manual ontology V2 (referenced as the YAML vocabulary published as part of this paper's deposit chain, `meta.version: '2.0'`). The Manual ontology V2 partitions the Manual's chapter-intrinsic content into entity types that the pipeline's coverage-analysis stage uses as the backbone for the three-way routing taxonomy of §7. The Manual ontology V2 is broader than AppSec Core v1: AppSec Core v1 normalizes the engineering substance of heterogeneous AppSec sources (objectives, practices, mechanisms, artifacts at L2 risk level in the Manual's risk convention), while Manual ontology V2 also covers Manual-intrinsic content that is out-of-scope for AppSec Core (governance, organisational, regulatory-overlay material). The pipeline's routing decisions surface this asymmetry per chapter (§7).

The relationship between AppSec Core v1 and Manual ontology V2 is the subject of separate future-work streams registered in §10: a formal alignment between the two ontologies (Stream 1) and a formal apparatus for Manual ontology V2 itself (Stream 2). For the pipeline reported in this paper, Manual ontology V2 is consumed at its frozen version as the chapter-content backbone, and the routing taxonomy operates over the intersection and complement of the two ontologies at the cycle's freeze tag.

---

3. Pipeline Architecture

The pipeline composes seven stages from external-source ingest through cycle-closure deposit chain. Figure 1 shows the architecture; Sections 3.1–3.3 specify the stages, the architectural surfaces emitted by the V1 ontology, and the integration points at which the V1 ontology binds into the pipeline.

3.1 Pipeline stages

Stage 1 — External-source ingest. The pipeline accepts heterogeneous normative security-engineering sources as input. The 31-source corpus consumed by this paper is the cycle-close substrate established in [5]: 26 baseline sources from the first-wave compilation work and 5 AI/ML extension sources (MITRE ATLAS, OWASP LLM Top 10, OWASP ML Top 10, NIST AI 100-2 e2025, NIST AI RMF 1.0).

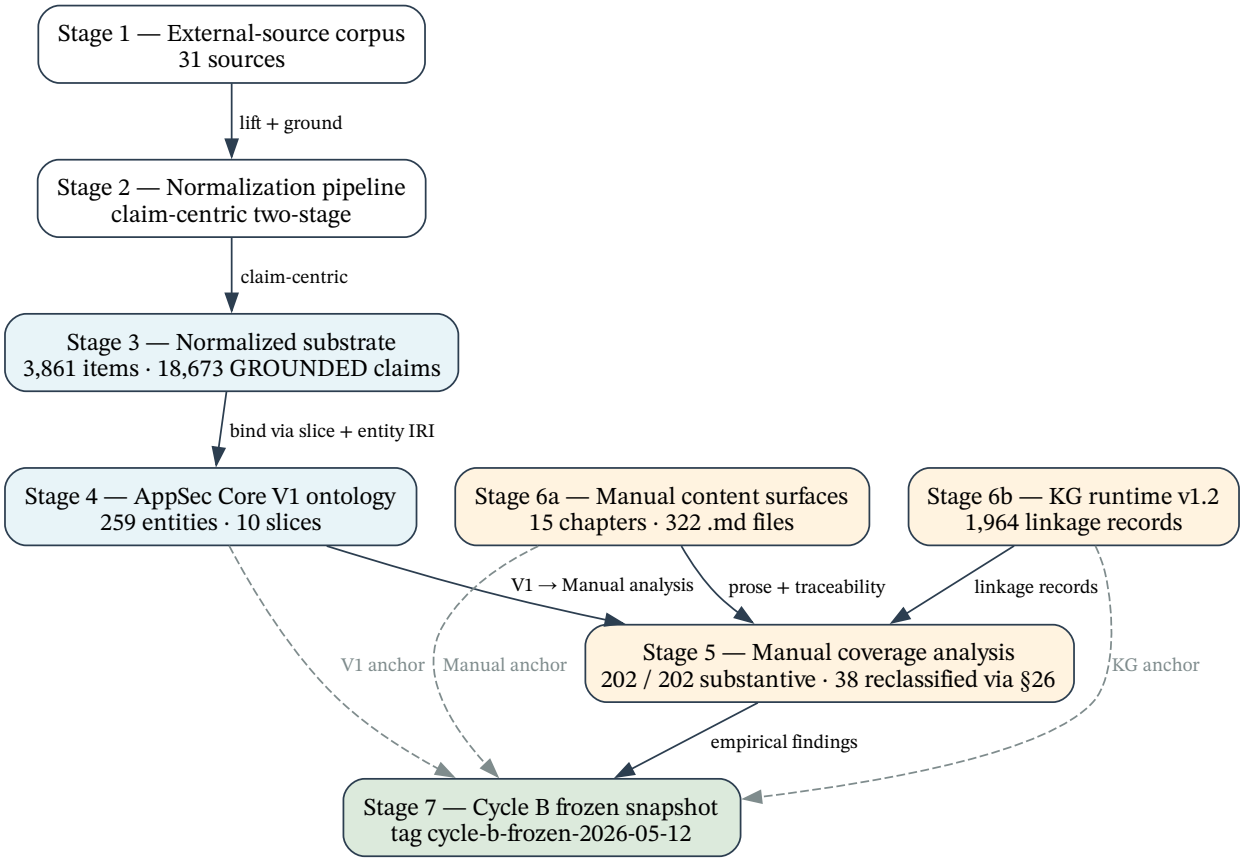

Figure 1: Pipeline Primitive Architecture. External-source corpus (31 sources) is normalized via the claim-centric two-stage pipeline into the normalized substrate (3,861 items, 18,673 GROUNDED claims). The normalized substrate binds against the AppSec Core V1 ontology (259 entities across 10 slices). The Manual coverage-analysis stage classifies per-entity Manual coverage and routes closure through three mechanisms (Section 6). Manual content surfaces and the KG runtime v1.2 (1,964 linkage records) are deposited at the cycle's closure state `cycle-b-frozen-2026-05-12` (Section 11).

Stage 2 — Normalization. Sources are lifted into a per-source claim-centric representation and grounded against the v1 ontology through a two-stage normalization pipeline (lifting followed by SBERT-based similarity grounding) detailed in [5].

Stage 3 — Normalized substrate. Stage 2 emits a normalized substrate published as part of [5]'s deposit chain (SUPPLIER SHA-256 `596783ed…`), comprising 3,861 source items (2,873 GROUNDED + 988 LabDepthPending) and 18,673 GROUNDED claims. Per-item GROUNDED rate is 74.4%; LabDepthPending items emit no claims by design.

Stage 4 — V1 ontology binding. The substrate claims bind against the AppSec Core V1 ontology published by [4], comprising 10 methodological slices (ASC-01 through ASC-10) and 259 typed instances (75 Control Objectives, 69 Practices, 58 Mechanisms, 57 Artifacts). At the cycle's closure, 202 of 259 entities (78.0%) receive at least one GROUNDED claim from the substrate; the 57 Artifact entities are not bound by the current substrate (a design choice of the normalization pipeline reported in [5]), and are classified as scope-exclusion in the coverage analysis of Stage 5.

Stage 5 — Manual coverage analysis. The V1-bound substrate is cross-referenced against the practitioner Manual to produce a per-entity coverage classification. A per-entity source map records, per V1 entity, the external sources contributing GROUNDED claims; coverage analysis then determines which Manual chapter authors each entity, with what evidence strength, and under which §26 methodology label (Section 4). The per-entity source map is published as part of this paper's deposit chain (Section 11.5).

Stage 6 — Manual content surfaces and KG compilation. The Manual is updated where coverage analysis surfaces closure actions; the knowledge graph runtime is re-compiled against the updated Manual. At the cycle's closure the Manual presents 15 chapters across four document families (traceability tables, achievable-maturity progressions, threat-mitigation catalogues, and review checklists), 322 markdown files total. The knowledge graph runtime v1.2 surfaces 1,964 Manual-to-ontology linkage records: 1,105 traceability records (5-section schema with §26 methodology labels), 336 maturity-progression records (SAMM v2.1 + DSOMM + SLSA per §26 Section 4 discipline), and 523 threat-mitigation records (Manual + CAPEC primary, CWE supporting). The Manual content surfaces and the knowledge graph runtime are published as part of this paper's deposit chain (Section 11.5).

Stage 7 — Joint closure pinning and public-deposit mirroring. Cycle closure produces a joint snapshot composed of the four pipeline outputs — the Manual content (Stage 6), the knowledge graph runtime v1.2 (Stage 6), the substrate-grounding evidence (Stage 3), and the per-stage closure brief — pinned together at the cycle's closure ledger anchor (`cycle-b-frozen-2026-05-12`). The joint snapshot is mirrored to the public deposits enumerated in Section 11.5.

3.2 V1 ontology architectural surfaces

Two version identifiers operate at the cycle and are distinct: AppSec Core V1 ontology at the **`v1.1-fair-baseline`** tag is the ontology artefact published by [4] (the OWL/Turtle bounded export, SHACL apparatus, embeddings release, and ontology metadata); knowledge graph runtime v1.2 is the downstream consumer surface compiled at the cycle (Section 9.2, Section 9.4), introducing additive consumer-contract surfaces (5-section traceability schema with §26 methodology labels; maturity-progression and threat-mitigation linkage families) over v1.1 without modifying the ontology artefact. The runtime's v1.2 designation refers to the consumer-contract evolution, not to an ontology version revision; AppSec Core V1.1 remains the ontology under consumption throughout the cycle. Subsequent references to "v1.1" denote the ontology at its FAIR-baseline tag and to "v1.2" denote the consumer-contract surface unless otherwise noted.

The V1 ontology published by [4] emits four formal artefacts that downstream pipeline stages bind to. The OWL/Turtle bounded export carries 1,970 data triples comprising the populated graph plus FAIR-baseline ontology metadata (license, creator, dates, publisher, citation, preferred-prefix, versionIRI, logo, status); class-level disjointness is declared via a single `owl:AllDisjointClasses` axiom over the five first-class entities. The SHACL apparatus composes six schema-derived NodeShapes (one per entity type plus the EvidencePattern declarative class) with five hand-maintained consumer-conformance Claim shapes (per Section 4 of [4]), totalling 396 shape triples; both `pyshacl 0.31.0` and the in-house bounded validator report `conforms=True` with zero violations at v1.1. The embeddings v1.1 release publishes a 212-entity augmented text corpus encoded as a 384-dimensional float32 L2-normalised NPZ tensor under a pinned encoder (`sentence-transformers/all-MiniLM-L6-v2` at HuggingFace revision `c9745ed1…`) with bit-identical reproducibility guaranteed across the 11-dimension build environment reported in Section 11. The substrate grounding is the downstream consumer side of the binding (reported by [5]), emitting 14,479 `ac:Claim` instances validated against the apparatus. The 14,479 apparatus-validated `ac:Claim` instances are a subset of the 18,673 GROUNDED claims emitted at substrate emission (Stage 3, reported by [5]); the residual 4,194 claims are substrate-internal at the cycle's closure (their per-source provenance preserved in the normalized substrate without apparatus-side `ac:Claim` materialisation, reflecting the per-claim filtering that the apparatus-validation stage applies under [4]'s SHACL conformance discipline).

3.3 V1 ontology integration points

The V1 ontology binds into the pipeline at six integration points: (a) source normalization (Stage 2) references V1 entity IRIs as the grounding target for per-source `*_lifted.jsonl` outputs; (b) similarity grounding (Stage 2, claim-centric Pipeline 2) consumes the embeddings v1.1 NPZ for source-item → V1 entity top-1 vote computation; (c) claim emission (Stage 3) produces `ac:Claim` triples carrying `target_core_entity` references to V1 IRIs, validated against the apparatus; (d) Manual coverage analysis (Stage 5) consumes the V1 entity index for the per-entity source map; (e) Manual ontology V2 (Section 2.4) overlays V1 entity identifiers as anchor points for Manual cross-reference resolution (e.g., a Manual VAL-008 requirement resolves to ACO-IVF-008 under the V1 typing); (f) KG runtime references (Stage 6) consume V1 entity identifiers and stable cross-entity relations for retrieval-surface composition. Future-work formal alignment between AppSec Core V1 and Manual ontology V2 (Stream 1 in Section 10) would substitute the current overlay binding with a declared SKOS/EDOAL alignment artefact.

---

4. Method

The pipeline operates under three methodological disciplines that together govern how per-entity coverage is classified, how each Manual document family is read, and how the two ontology layers (AppSec Core V1 and Manual ontology V2) interact at the traceability surface. The disciplines are canonical artefacts of the Manual: they live in §26 of the Manual's foundational canon (published as part of this paper's Manual deposit, Section 11.5) and are consumed by the pipeline as published methodological inputs, not authored by this paper.

4.1 Coverage classification vocabulary

The §26 canon defines six labels for the classification of per-entity coverage at the traceability surface. The labels are deterministic — each entity's coverage-classification output (initial detection followed by deterministic keyword-based refinement) maps to exactly one label per row:

Label	Trigger condition	Count
Explicit	V1 entity directly anchored to a Manual section via existing canonical document or concept mapping; pre-existed before the cycle's coverage-analysis pipeline	164
Semantic	Strong keyword evidence (≥3 distinct keywords with ≥5 total occurrences) in the chapter prose expected from the V1 slice anchor; coverage exists, but the traceability exposure was absent before the cycle	31
Partial	Strong keyword evidence only in chapter(s) not expected from the V1 slice anchor; coverage is cross-chapter; navigation pointer required	6
Gap	Weak or no keyword evidence anywhere; genuine authoring deficit; entity registered for the future-work register rather than authored under closure pressure	1
Scope boundary	Artifact entity (`ACA-*`); structural completeness held at the OWL level; the normalized substrate grounding deferred per pipeline scope (Section 3)	57

A sixth mechanism, Repair, is not a classification label but a re-emission discipline applied during the cycle's entity-first traceability regeneration: it re-writes the traceability tables so that previously hidden Semantic and Partial cases (37 entries at the cycle's closure) become visible at the traceability surface, without modifying the underlying Manual prose. Repair is the closure mechanism for Semantic and Partial labels (Section 6); the labels themselves classify the coverage state.

Aggregate at the V1 substantive surface (the 202 of 259 entities consumed by the normalized substrate grounding pipeline, excluding the 57 Artifact entities under Scope boundary): 164 Explicit + 31 Semantic + 6 Partial + 1 Gap = 202. Counts are recomputed from the canonical state at the cycle's closure anchor (published as part of this paper's deposit chain, Section 11.5); no intermediate-cycle scaling or delta-from-prior-state derivation is invoked.

The keyword-density threshold employed by Semantic / Partial / Gap classification (≥3 distinct keywords with ≥5 total occurrences) instantiates the threshold-based concept-instance matching pattern catalogued in the ontology-learning literature [14] and grounded in classical IR term-frequency methods [15]; the specific thresholds are calibrated to the §26 canon's reading discipline at the cycle's substrate scale and are not claimed as general.

4.2 Per-family reading discipline

The §26 canon §4 defines a distinct reading discipline for each of the Manual's three substantive document families:

Traceability tables (`<chapter>/canon/25-rastreabilidade.md`, 15 chapters; the Portuguese filename is the canonical artefact path in the Manual repository and is preserved as a path literal — "traceability" is used throughout this paper for the conceptual surface) answer the question *which external frameworks and sources does this chapter support, with what kind of coverage?* The output is an entity-first listing per row, with six columns: Manual ontology V2 anchor, Manual section anchor, Authority class, Source mode, §26 methodology label, and the external-source grounding evidence trail (Section 7 specifies the routing layers built atop this column set).

Achievable-maturity progressions (`<chapter>/achievable-maturity.md`, 14 chapters; chapter 00-fundamentos is excluded from this family) answer the question *if this chapter is implemented as written, what maturity posture is credibly attainable?* The reading discipline restricts maturity sources to SAMM v2.1 and DSOMM as primary; SLSA is conditional and applies only where build or integrity progression is meaningful; regulatory alignment is recorded out-of-maturity to avoid conflation between compliance posture and engineering maturity. Each row receives a §26 methodology label deterministically derived from a `confidence` threshold ($\geq$0.85 maps to Explicit; $\geq$0.65 to Semantic; $\geq$0.4 to Partial; below 0.4 to Gap).

Threat-mitigation catalogues (`<chapter>/canon/50-ameacas-mitigadas.md`, 14 chapters; chapter 00-fundamentos is excluded) answer the question *which threat families does this chapter mitigate, and is the mitigation strong, partial, or dependent on other chapters?* The reading discipline restricts threat-source authority: the Manual's own canonical surface and CAPEC are the primary anchors; CWE is supporting but does not substitute for a threat taxonomy. Each row receives a §26 label plus a mitigation-strength label drawn from a closed vocabulary (`forte`, `parcial`, `dependente_de_outros_capítulos`).

4.3 Ontology layer separation

The pipeline operates over two orthogonal ontology layers that coexist in the traceability tables without functional overlap. The §26 methodology layer classifies row-level claim quality through the six-label vocabulary of Section 4.1 and reading-discipline boundaries of Section 4.2; this is editorial discipline operating as meta-process. The Manual ontology V2 layer (the YAML vocabulary published as part of this paper's deposit chain, `meta.version: '2.0'`) provides the entity structural model: fifteen entity types (Requirement, Control, Practice, Artifact, Threat, Concept, Mechanism, Pattern, AntiPattern, Signal, and others) each carrying an Authority class (normative, editorial, semantic, operational, external), a Source mode (explicit, derived, scored, heuristic, runtime), and a Confidence model (deterministic, bounded, probabilistic). Each traceability row carries both layers' columns: the V2 layer answers a structural question (what type of entity, with what authority); the §26 layer answers an evidential question (with what claim quality).

The two layers operate at different granularities: §26 labels are per-row classifications driven by claim evidence; V2 anchors are per-entity structural typings. The pipeline produces both, and a downstream consumer of the Manual + KG joint snapshot can route on either dimension independently. Section 7 reports the empirical surface — the per-chapter routing data — that this layer separation makes visible.

4.4 Anti-rush content discipline

Where the coverage classification surfaces a Gap label (Section 4.1), the cycle's discipline is to register the entity for the future-work surface (Section 10) rather than author Manual prose under closure pressure. At the cycle's closure, this discipline produced one such registration (`ACM-IVF-004`, the centralised error-translation-and-redaction mechanism). The discipline is asymmetric in favour of registered-deferral over rushed-authoring: a registered gap is a tractable input to a subsequent cycle, whereas hurried prose authoring under closure pressure carries content-quality risk that the §26 reading discipline cannot retroactively absorb. The cycle does not claim that the registered gap will be addressed in any particular subsequent cycle; the claim is only that the cycle did not produce authored content for that entity, and the future-work surface preserves visibility for downstream planning. The discipline is also tolerant of false-positive Gap classifications: because the §26 refinement carries no independent validation surface (Section 10.7), a registered Gap entry may resolve to Semantic or Partial at a subsequent cycle under a broader keyword vocabulary or an external validation reading, and the future-work surface accommodates that reclassification by treating registered entries as candidates for re-examination rather than committed authoring backlog.

---

5. Cycle Execution

The cycle reported in this paper is one instantiation of the pipeline (Section 3) against the cycle-entry substrate. The cycle is internally identified as Cycle B in the programme's cycle ledger and is pinned at the closure ledger anchor `cycle-b-frozen-2026-05-12`; Cycle A is the prior cycle that closed the v0→v1 ontology transition reported in [4] and [5], anchored at the figshare bundle `cycle-a-frozen-2026-05-08` and consumed here as the cycle-entry substrate. The second-execution framing of Section 1.2 and the between-execution comparison of Section 8 reference this Cycle A → Cycle B sequence; cycle-labelling beyond Cycle B is reserved for subsequent pipeline re-executions (Section 10.5). The cycle executed in six stages spanning the closure period 2026-05-09 to 2026-05-12; each stage's output is traced to a specific pipeline-functional role (Table 1). The stages are described here at the granularity of the pipeline rather than as cycle-internal iteration narrative: the substantive observation is that the seven-stage pipeline of Section 3 was exercised end-to-end against the V1 ontology, the 31-source substrate, and the practitioner Manual, producing the joint Manual + KG snapshot ratified at the cycle closure anchor. Per-stage canonical commit anchors are reported in the section's appendix for reproducibility (Section 5.7) without surfacing them in the main narrative.

5.1 Substrate-grounded traceability regeneration

The cycle's first stage replaced fifteen pre-existing traceability tables, each of which had been derived under an earlier substrate generation, with tables regenerated directly from the normalized substrate from the prior cycle GROUNDED claims (Section 3.1, Stage 3). The regeneration produced an explicit external-source → AppSec Core entity mapping per row, drawing from the 31-source corpus. The fifteen chapters thereby received a uniform traceability backbone aligned to the cycle's substrate state at entry.

5.2 Manual content extension for AI/ML domain coverage

The cycle's second stage extended Manual prose for five chapters (`03-threat-modeling`, `04-arquitetura-segura`, `05-dependencias-sbom-sca`, `06-desenvolvimento-seguro`, `12-monitorizacao-operacoes`) with content drawn from the five AI/ML extension sources newly admitted to the cycle's corpus (MITRE ATLAS, NIST AI 100-2 e2025, NIST AI RMF 1.0, OWASP LLM Top 10, OWASP ML Top 10). The extended prose addresses adversarial machine learning, LLM-application security, and

AI risk governance topics that had no Manual surface before the extension. The extension is the only stage at which the Manual prose corpus grew during the cycle; subsequent stages operate over the prose corpus as a fixed input.

5.3 Entity-first traceability re-emission

The cycle's third stage re-emitted the fifteen traceability tables under an entity-first row layout (V1 entity per row, with external-source columns), replacing the source-first layout produced by the regeneration of Section 5.1. The re-emission did not modify Manual prose; it surfaced previously hidden Semantic and Partial coverage cases (Section 4.1) by exposing per-V1-entity traceability rows that had been absent under the source-first layout. The re-emission is the operational realization of the Repair mechanism (Section 4.1).

5.4 Three-way routing exposure

The cycle's fourth stage refactored each traceability table into four tabular sections (Core-mapped, Manual-only, future-work, Out-of-AppSec), exposing the three-way routing taxonomy as a per-chapter structural property of the traceability surface. The routing categories were already implicit in the entity-first layout of Section 5.3; the refactor renders them explicit and table-level navigable. Section 7 reports the per-chapter routing outcomes that this stage made surfaceable.

5.5 Manual ontology V2 vocabulary integration

The cycle's fifth stage refreshed the §26 methodology canon to incorporate the Manual ontology V2 vocabulary atop the existing V1 substrate-grounded bindings. The integration is overlay-shaped (Section 4.3): the V2 vocabulary columns (Authority class, Source mode, V2 entity anchor) were added to the traceability tables alongside the existing V1 substrate-evidence chain, without modifying the underlying V1 bindings. The §26 canon was refreshed in the same stage to reflect the V2-integrated reading discipline (Section 4.2).

5.6 Methodology vocabulary propagation to maturity and threat families

The cycle's sixth stage applied the §26 methodology vocabulary beyond traceability tables to the two additional document families introduced in Section 4.2: achievable-maturity progressions (`<chapter>/achievable-maturity.md`, populated for fourteen chapters; chapter 00-fundamentos retains its baseline-only structure) and threat-mitigation catalogues (`<chapter>/canon/50-ameacas-mitigadas.md`, populated for fourteen chapters under the same exception). The stage propagated the substrate-evidence chain into 336 maturity-progression records and 523 threat-mitigation records (Section 9 details the KG runtime surface counts).

The cycle's closure followed Stage 5.6 with the closure ledger annotated tag `cycle-b-frozen-2026-05-12` (Section 9), pinning the cycle's Manual + KG joint snapshot across the four artefact-class origin repositories ahead of mirroring to the public deposit surfaces (Section 11).

5.7 Reproducibility — per-stage canonical commits

For reproducibility audit, each stage of Sections 5.1–5.6 is anchored at a specific commit in the Manual repository:

Stage	Section	Manual repository commit
Substrate-grounded traceability regeneration	§5.1	`94a757b0`
Manual content extension (AI/ML)	§5.2	`b1d129f0`

Stage	Section	Manual repository commit
Entity-first traceability re-emission	§5.3	`71b9c029`
Three-way routing exposure	§5.4	`16dfa5ae`
Manual ontology V2 vocabulary integration	§5.5	`a9e70c98`
Methodology vocabulary propagation to maturity and threat families	§5.6	`455124a1`
Joint closure	§5.6 → §9	`cycle-b-frozen-2026-05-12` (closure ledger anchor, Section 9.1)

The per-stage commits are surfaced under the cycle's closure tag in this paper's deposit chain (Section 11.5); peer tags pin the substrate and ontology state consumed at each stage to the deposit chains of [4] and [5].

---

6. Closure Mechanisms Demonstrated

The cycle's coverage-analysis stage (Section 3.1, Stage 5; Section 5.3 entity-first re-emission) surfaced 38 entries that the initial coverage detection had flagged as content-gap candidates at the V1 substantive surface. A subsequent deterministic keyword-based refinement step, applied during the traceability re-emission stage and the §26 vocabulary integration, classified these 38 entries into three closure mechanisms (Table 2). The mechanisms operationalize the Repair discipline of Section 4.1 for the Semantic and Partial label classes; the Gap label class is treated under the anti-rush content discipline of Section 4.4.

6.1 Three closure mechanisms

Closure mechanism	Count	% of 38	§26 label	Prose authoring scope
Traceability exposure of pre-existing prose	31	81.6%	Semantic	Zero new prose; traceability row added with keyword evidence trail
Cross-chapter pointer added	6	15.8%	Partial	Zero new prose; navigation pointer to cross-chapter coverage
Future-work register	1	2.6%	Gap	Deferred per Section 4.4 anti-rush discipline; registered in Section 10

Table 2. Closure-mechanism distribution at the V1 substantive surface (38 non-Explicit entries; excludes the 164 Explicit entries already covered before the cycle and the 57 Artifact entities under Scope boundary). Counts recomputed from the canonical state at this paper's gap-classification output (Sec-

tion 11.5).

6.2 Traceability exposure of pre-existing prose

Thirty-one of the thirty-eight entries (81.6%) closed under the Semantic label: Manual prose addressing each entity already existed in the chapter expected from the V1 slice anchor, but the traceability surface had not exposed the entity-level mapping before the cycle. The closure mechanism added a per-entity traceability row anchored to the chapter, with a keyword evidence trail captured in the `manual_section_anchor` field of the KG record (an excerpt of the form `chapter prose (kw1, kw2, kw3 kws verified)` recording the deterministic keyword match against the chapter prose). The mechanism authored no new Manual prose; the closure consists entirely of traceability-surface exposure of content that already existed.

The KG runtime v1.2 (published as part of this paper's KG deposit, Section 11.5) carries uniform `methodology_label: "Semântico"` across these thirty-one records; the keyword evidence trail in the `manual_section_anchor` field is the empirical signature of the mechanism. The thirty-one entities span eight of the ten slices (ASC-01, ASC-02, ASC-03, ASC-04, ASC-05, ASC-07, ASC-08, ASC-09, ASC-10).

6.3 Cross-chapter pointer added

Six of the thirty-eight entries (15.8%) closed under the Partial label: prose addressing each entity existed in the Manual, but in one or more chapters distinct from the chapter expected from the V1 slice anchor. The closure mechanism added a cross-chapter pointer in the entity's traceability row, listing the chapters where the entity is actually addressed; the mechanism did not relocate Manual prose. The six entries span the architecture (ACO-ATB, ACO-IAT, ACO-ITS) and secure-development (ACO-IVF, ACO-SPC) slices and are documented at this paper's traceability linkage records (Section 11.5) with `methodology_label: "Parcial"` and the cross-chapter pointers in the `manual_section_anchor` field.

6.4 Future-work register

One of the thirty-eight entries (2.6%) carried the Gap label: the centralised error-translation-and-redaction mechanism `ACM-IVF-004` was identified as a content-authoring deficit candidate, with weak or no keyword evidence in any chapter under the §26 deterministic refinement (Section 4.1). Per the anti-rush content discipline of Section 4.4, the entity was registered for the future-work surface (Section 10) rather than authored under the cycle's closure pressure. The KG record carries `methodology_label: "Gap"` and `manual_section_anchor: "□ future-work (P8 §10)"`; the entity appears in the manifest's future-work entries register at `data/publish/runtime/v1/v1_manifest.json`. The Gap classification is reported as the deterministic refinement output and not as independently verified absence: the keyword-detection threshold (Section 4.1) can fail to match prose that addresses the entity under terminology absent from the keyword profile, so `ACM-IVF-004` is registered under that caveat (Section 10.7) and re-examination at subsequent cycles, or under an external validation reading, may reclassify the entry as Semantic or Partial if substantive coverage exists under unmatched vocabulary.

6.5 Refinement outcome: detection vs content deficit

The substantive empirical finding of the cycle is not the closure ratio, which is tautologically resolvable under the pipeline's three-mechanism vocabulary, but the redistribution of the 38 initially-detected gaps under deterministic keyword-based refinement. Of the 38 entries that initial coverage detection flagged as candidate content-authoring deficits at the V1 substantive surface, refinement reclassifies 31 (81.6%) as Semantic — Manual prose addressing the entity already existed in the expected chapter, and the

deficit is in the traceability surface rather than the chapter prose. A further 6 (15.8%) are reclassified as Partial — prose exists but in a chapter different from the V1 slice anchor, a navigation deficit rather than a content deficit. Only 1 of 38 (2.6%; `ACM-IVF-004`) survives refinement as a Gap — a genuine content-authoring deficit registered for the future-work surface (Section 10.1) under the anti-rush discipline (Section 4.4).

The finding, in honest framing, is that detection-stage gap counts substantially overstate content-authoring need at this substrate scale: 37 of 38 candidate gaps were false positives of the detection step in the sense that prose covering the entity already existed somewhere in the Manual corpus (in the expected chapter for 31; cross-chapter for 6). The closure intervention is correspondingly minimal — 31 traceability rows exposed, 6 cross-chapter pointers added, 1 entity registered for future authoring — and no Manual prose was modified under the cycle's closure pressure.

This finding — that detection over-reports gaps and refinement reveals predominantly traceability deficits — is contrasted against the earlier 5-source execution of [2] in Section 8, where the gap-class distribution under the earlier programme's three-class taxonomy showed a substantially higher genuine-content-gap share. The detection-followed-by-refinement pattern instantiates the recall-precision trade-off long-documented in the IR literature [16]: the detection step is deliberately set to high recall (over-inclusive), and the deterministic keyword threshold of Section 4.1 trades that against precision at the refinement stage; the empirical surface at this substrate scale is the resulting 81.6% Semantic / 15.8% Partial / 2.6% Gap redistribution.

6.6 KG runtime verification of closure

The KG runtime v1.2 surfaces all 38 entries in the `manual_rastreabilidade.jsonl` substrate-landing records, providing a verifiable closure trail per entity. Across the full KG v1.2 surface (1,964 records spanning traceability, maturity-progression, and threat-mitigation linkages; Section 9), the §26 methodology label distribution is recorded at `data/publish/runtime/v1/v1_manifest.json` under `methodology_label_distribution`: 565 Explicit, 31 Semantic, 6 Partial, 1 Gap. The four §26 substantive labels sum to 603 records of the 1,964-record surface; the residual 1,361 records carry no substantive §26 label because they are authored at granularities other than per-V1-entity at the traceability surface — chapter-level or section-level navigation rows, maturity-progression records whose `confidence` falls into the Partial-or-below bands without a per-V1-entity anchor, and threat-mitigation records anchored at the threat-family granularity rather than the V1 entity granularity. The 38 cycle-closure entries (31 Semantic + 6 Partial + 1 Gap) account for the entire non-Explicit population at the per-V1-entity traceability surface; the 565 Explicit count aggregates Explicit-labelled records across all three document families and exceeds the 164 Explicit substantive-V1 entities counted at the traceability-only surface (Section 4.1) because maturity-progression and threat-mitigation records also carry §26 labels at the granularities they declare.

---

7. Three-Way Routing per Chapter

The three-way routing taxonomy — Core-mapped, Manual-only, and Out-of-AppSec — is the architectural surface that the cycle's traceability re-emission and four-section tabular refactor (Sections 5.3 and 5.4) made navigable per chapter. The taxonomy is consumed by the Manual reader through the four-section traceability layout and by the KG runtime through the per-chapter linkage records (Section 9). This section reports the per-chapter routing surface at the cycle's closure and surfaces the substantive empirical asymmetry it makes visible.

7.1 Per-chapter routing surface

Table 3 reports the per-chapter routing distribution at the cycle's closure, drawing from the canonical Manual state at the closure-tag checkout of the public Manual repository (`SbD-ToE/sbd-toe-manual@cycle-b-frozen-2026-05-12`, under `docs/sbd-toe/010-sbd-manual/`). The V2-entity column counts Manual ontology V2 entities indexed per chapter; the V1-substantive column counts V1 Control Objectives, Practices, and Mechanisms anchored to the chapter via the slice-to-chapter map (Artifacts excluded under Scope boundary per Section 4.1); the Semantic / Partial / Gap columns count the three closure-mechanism populations of Section 6; the Manual-only and Out-of-AppSec columns count Manual sections without V1 overlay anchoring per Section 7.3 and Section 7.4 inventories.

Chapter	V2 entities	V1 substantive	Semantic	Partial	Gap	Manual-only sections	Out-of-AppSec sections	Future-work
00-fundamentos	0	0	0	0	0	0	0	0
01-classificação-aplicações	40	0	0	0	0	0	0	0
02-requisitos-segurança	122	0	0	0	0	0	0	0
03-threat-modeling	46	25	4	0	0	3	4	0
04-arquitetura-segura	66	56	5	3	0	4	4	0
05-dependências-sbom-sca	25	20	3	0	0	3	2	0
06-desenvolvimento-seguro	25	37	3	3	1	3	3	1
07-cicd-seguro	81	0	0	0	0	0	0	0
08-iac-infraestrutura	87	0	0	0	0	0	0	0
09-containers-imagens	33	0	0	0	0	0	0	0
10-testes-segurança	29	19	1	0	0	3	2	0
11-deploy-seguro	30	27	11	0	0	2	1	0
12-monitorização-operações	68	18	4	0	0	3	2	0

Chapter	V2 entities	V1 substantive	Semantic	Partial	Gap	Manual-only sections	Out-of-AppSec sections	Future-work
13-formação-onboarding	79	0	0	0	0	0	0	0
14-governança-contratação	76	0	0	0	0	0	0	0
Total	807	202	31	6	1	21	18	1

Table 3. Per-chapter routing distribution at the cycle's closure. Chapter names preserve canonical Portuguese identifiers as they appear in the Manual repository file paths (`<chapter>/` directories), which downstream readers use to navigate the Manual. Bold rows mark five chapters where the Manual ontology V2 entity surface substantially exceeds the AppSec Core V1 overlay coverage; Section 7.2 reports the substantive empirical asymmetry that this surface makes visible.

7.2 V1-overlay-minimal chapters and the V2/V1 asymmetry

Five of the fifteen Manual chapters carry zero V1 substantive overlay entities under the slice-to-chapter mapping at the cycle's closure: chapter 02 (security requirements), chapter 07 (CI/CD), chapter 08 (infrastructure-as-code), chapter 13 (onboarding/training), and chapter 14 (governance and contracting). The Manual ontology V2 indexes $122 + 81 + 87 + 79 + 76 = 445$ entities across these five chapters at the cycle's closure; the V1 overlay indexes zero. Three additional chapters — 00-fundamentos (foundational concepts), 01-classification, and 09-containers — likewise carry zero V1 substantive overlay; together with the five V1-overlay-minimal chapters, eight of fifteen chapters constitute the Manual-only territory at the cycle's closure: chapters whose Manual ontology V2 surface is substantively populated but for which AppSec Core V1 does not contribute overlay entities under the current slice partition.

The empirical asymmetry — V1 indexing 259 entities concentrated in seven chapters; V2 indexing 807 entities spread across all fifteen chapters with five V1-overlay-minimal chapters carrying 445 V2 entities alone — admits two readings that the cycle's pipeline outputs alone do not arbitrate between. Under the first reading, AppSec Core V1 is bounded by design to the engineering-substance core of the Manual: the eight V1-overlay-minimal chapters carry organisational, methodological, and governance content (security-requirements management, CI/CD discipline, infrastructure-as-code policy, container image governance, onboarding curricula, governance and contracting frameworks, foundational concepts, and application classification) that lies outside the V1 ontology's design boundary as currently scoped, and the asymmetry is therefore a deliberate scope-boundary expression rather than a deficit. Under the second reading, the asymmetry surfaces candidate V2-or-V3 expansion territory: chapters 02 (security-requirements management) and 07 (CI/CD), in particular, fall within the typical scope of application-security ontologies, and the absence of V1 overlay entities there may be an evolvable scope gap rather than an intentional boundary; chapters 13 (training), 14 (governance and contracting), and 00 (foundational concepts) are more plausibly out-of-scope-by-design. The cycle's pipeline does not resolve which reading applies per chapter — that resolution requires an ontology-scope review independent of the cycle's grounding analysis, registered for subsequent ontology-version work alongside the alignment and verification streams of Sections 10.2 and 10.4. Independently of the scope-boundary question, the asymmetry confirms a second observation about the Manual baseline: the Manual editorial baseline preserves substantive richness independent of the V1 overlay; the cycle's pipeline added

a focused substrate-grounded mediation layer (V1) on top of a Manual that already had a broader V2 entity surface, rather than reducing the Manual's content to the V1 substrate scope.

7.3 Manual-only sections — declared inventory

Twenty-one Manual sections across seven chapters carry Manual-only coverage: content with V2 entity anchoring and external-source grounding (SAMM, DSOMM, regulatory frameworks, organisational policies) but no V1 overlay. The inventory at the cycle is concentrated in the achievable-maturity files (`<chapter>/achievable-maturity.md`), the policies-relevant index files (`<chapter>/policies-relevantes.md`), and the addon files covering KPIs/metrics, vocabulary glossaries, and team guidelines:

Chapter	Count	Manual-only V2 anchor types in use
03-threat-modeling	3	MaturityMapping (SAMM/DSOMM), PolicyReference, KPI-metric overlay
04-arquitetura-segura	4	MaturityMapping, PolicyReference, ExternalFramework (vocabulary glossary), KPI-metric overlay
05-dependências-sbom-sca	3	MaturityMapping, PolicyReference, KPI-metric overlay
06-desenvolvimento-seguro	3	MaturityMapping, PolicyReference, OverlayPlaybook (team guidelines)
10-testes-segurança	3	MaturityMapping, PolicyReference, meta-testing component catalogue
11-deploy-seguro	2	MaturityMapping, PolicyReference
12-monitorização-operações	3	MaturityMapping, PolicyReference, SIEM-integration overlay

The dominant V2 anchor type at Manual-only coverage is `MaturityMapping`, reflecting the achievable-maturity document family's role as the primary maturity-progression surface (Section 4.2 reading discipline). The pattern is consistent with the V2 ontology's broader scope: maturity progression, policy reference, and KPI overlay material are first-class V2 entities that the V1 ontology, by design, does not type.

7.4 Out-of-AppSec sections — declared inventory

Eighteen Manual sections across seven chapters carry Out-of-AppSec coverage: pure editorial content (worked examples, narratives, ADR templates, vendor-specific tooling integration) without external-source grounding and without first-class V2 entity anchoring beyond a `DocumentUnit` or `UserStory` placeholder:

Chapter	Count	Representative content
03-threat-modeling	4	Privacy threat-modeling example; STRIDE worked examples; process narrative; tooling integration
04-arquitetura-segura	4	Lifecycle user-story; practical case set; reference diagrams; ADR examples
05-dependências-sbom-sca	2	CI/CD integration examples; origin-record examples
06-desenvolvimento-seguro	3	Code best-practice catalogue; exception narratives; semantic-annotation examples
10-testes-segurança	2	Penetration-testing narrative; AI-in-testing operational guidance
11-deploy-seguro	1	Incident-response playbook examples
12-monitorização-operações	2	Incident-response case set; event examples

Out-of-AppSec content is the editorial layer that the pipeline does not route through V1 or V2 ontology typing; it is preserved as the Manual's editorial corpus and surfaced in the traceability tables under the dedicated four-section layout introduced by Section 5.4.

7.5 Pipeline-primitive thesis reinforcement

The three-way routing surface reinforces the paper's pipeline-primitive thesis (Section 1.2). Each routing destination is a first-class output of the pipeline rather than a residual: the four-section tabular layout introduced in Section 5.4 explicitly allocates Manual table space to each of the three destinations (Core-mapped, Manual-only, Out-of-AppSec) plus the Out-of-Manual fourth section, treating each as a navigable surface that the pipeline produces by design, not as content the pipeline failed to type. The 21 Manual-only sections and 18 Out-of-AppSec sections are substantially populated — 39 out-of-V1-scope sections distributed across the seven V1-anchored chapters, an average of approximately 5.6 per chapter — and the eight Manual-only chapters carry the bulk of the Manual's V2 entity surface (445 of 807 V2 entities concentrated in five V1-overlay-minimal chapters). The cycle's pipeline composes the V1 substrate-grounded overlay on top of this broader V2 surface coherently: V1 contributes the engineering-substance subset routable through the normalized substrate; V2 retains the broader chapter-intrinsic surface that the Manual already authored; the three-way routing taxonomy makes both contributions visible per chapter without conflating them. A future iteration of the pipeline applied at a different programme scope — for instance, a Manual covering distinct domain families, or an evolved V1 partition that absorbs organisational territory — would re-execute the same routing taxonomy against the new substrate, producing the same three-way routing surface against an updated entity distribution.

8. Between-Execution Gap-Class Redistribution

The cycle's pipeline produces gap-classification outputs that can be contrasted against the earlier programme execution reported in [2] at five-source first-wave scale. The contrast is conducted as a between-execution gap-class redistribution analysis at the close states of the two executions, without scaling either set of counts to a common denominator and without delta-from-intermediate-state archaeology. Both sets of counts are recomputed from their respective canonical states at the close anchors: the present cycle's counts from this paper's gap-classification output (Section 11.5); the five-source execution's counts from the values reported in [2] at the published-paper scope. The two executions differ along three confounded axes — substrate corpus, ontology scope, and methodology vocabulary — that Section 8.5 makes explicit; the redistribution is reported descriptively, not as a controlled re-measurement.

8.1 V1-scale gap classification at cycle closure

At the cycle-close V1 substantive surface (the 202 V1 Control Objectives, Practices, and Mechanisms consumed by the normalized substrate grounding pipeline; the 57 Artifact entities are excluded under Scope boundary per Section 4.1), 164 entries are classified Explicit (covered through pre-existing canonical mapping before the cycle), 31 are classified Semantic, 6 are classified Partial, and 1 is classified Gap. The 38 entries classified non-Explicit constitute the cycle's refined gap-class population.

8.2 Five-source execution reference

The earlier programme execution reported in [2] established a gap-class distribution at first-wave five-source corpus scope. At that execution's close state, the per-class distribution of apparent gaps was: 22 claim-gap entries (61% of the 36 apparent gaps at that scope), 7 cross-reference-gap entries (19%), 4 content-gap entries (11%), and 3 scope-exclusion entries (8%). The earlier execution's class definitions correspond closely to the present cycle's refined classification labels: the [2] *claim-gap* class corresponds to Semantic (coverage present, traceability exposure absent); *cross-reference-gap* corresponds to Partial; *content-gap* corresponds to Gap.

8.3 Gap-class redistribution

Gap class	[2] five-source execution	the present cycle's V1 scope	Δ proportion
claim-gap (Semantic)	61% (22 / 36)	82% (31 / 38)	+21 pp
cross-reference-gap (Partial)	19% (7 / 36)	16% (6 / 38)	−3 pp
content-gap (Gap)	11% (4 / 36)	3% (1 / 38)	−8 pp
scope-exclusion	8% (3 / 36)	(57 Artifact entities; reported at V1 scope under Section 4.1 Scope boundary)	not directly comparable

Table 4. Gap-class redistribution between the [2] five-source execution and the present cycle's V1 scope. The Δ column should not be read as a controlled re-measurement under a single instrument: the §26 six-label vocabulary (Section 4.1) is a refinement of, not a replication of, the [2] three-class taxonomy, and the two executions differ along three confounded axes (corpus size, ontology scope, methodology vocabulary). The class correspondences (claim-gap ? Semantic; cross-reference-gap ? Partial; content-gap ? Gap) are conceptually close — both vocabularies separate traceability-surface deficits from cross-chapter pointers and from genuine content deficits — but are not strictly equivalent: the §26 vocabu-

lary's keyword thresholds (Section 4.1) introduce a precision that the [2] classification did not encode, and Semantic at §26 admits cases at the same V1-slice-anchored chapter that the [2] *claim-gap* class admitted at broader granularity. The Δ values are therefore reported as qualitative redistribution under the refined vocabulary at the cycle's substrate scale, not as a controlled instrument re-measurement. The scope-exclusion row is reported descriptively only: the [2] earlier execution's three scope-exclusions are not directly comparable to the present cycle's 57 Artifact entities under Scope boundary, which arise from the normalization pipeline's design scope (Section 3.1 Stage 3) and operate at a different denominator (V1 entity surface rather than apparent-gap surface).

8.4 Substantive empirical finding

Two complementary movements characterise the present cycle's distribution relative to the five-source execution. First, the Semantic (claim-gap) share rises by 21 percentage points, from 61% at the earlier execution to 82% at the present cycle's closure. Second, the Gap (content-gap) share falls by 8 percentage points, from 11% to 3%. The Partial (cross-reference) share is approximately stable (−3 pp, within rounding noise at small absolute counts). Read together with the §6.5 finding, the redistribution reinforces the same observation under a between-execution lens: as the substrate scales from 5 to 31 sources and the Manual prose corpus matures under the cycle of [5], what the detection step flags as ambiguous resolves predominantly to a traceability-surface deficit rather than to a content-authoring deficit. At the present cycle, only one genuine content gap survives refinement (`ACM-IVF-004`, Section 6.4), registered for the future-work surface under §4.4's anti-rush discipline; the Manual prose authoring scope for cycle closure was zero.

The interpretation respects the pipeline architecture of Section 3: the traceability-exposure mechanism (Section 6.2) is exactly the closure mechanism that this gap profile makes available, and applies to 31 of the 38 non-Explicit entries with no Manual prose modification. The reading is descriptive of the present-cycle redistribution, not predictive: whether the same redistribution recurs at substantially different substrate compositions (Manual covering distinct domain families, ontology evolved beyond V1 partition cardinality, corpus dominated by AI/ML or regulatory authority classes) is an empirical question for subsequent cycles, not foreclosed by the cycle reported here.

8.5 Boundaries of the comparison

The contrast supports a between-execution redistribution claim, not a generalisation to substantially different programme states. The two executions operate over different corpora (five sources vs thirty-one sources), different ontology scopes (V0 first-wave entity surface vs V1 cycle-close entity surface), and different methodological vocabularies (the earlier execution's three-class taxonomy was refined by the present cycle's six-label §26 vocabulary, Section 4.1). The Δ values report the redistribution observed across these confounded axes; they do not isolate a single causal factor (corpus expansion, ontology evolution, methodology refinement). A future execution running the same pipeline against a different substrate composition — a different Manual or a substantially different external-source set — would produce a different gap-class profile; whether the redistribution pattern reported here generalises is an empirical question for those subsequent executions, not a claim of this paper.

---

9. Cycle Closure State and Public Deposits

The cycle's closure state is pinned at an internal-governance ledger anchor — the annotated tag `cycle-b-frozen-2026-05-12` — created under co-ordinated programme governance. The ledger anchor records, across the four artefact-class origin repositories that the pipeline consumes and produces, the

closure-commit pointers for the cycle state at which Sections 3 through 8 are reported. The ledger anchor is not the external citation form: only one of the four origin repositories (the public Manual repository `SbD-ToE/sbd-toe-manual`) is publicly accessible; the remaining three are not. The external citation surfaces that downstream consumers reference are the public deposits enumerated in Section 11 — the public research repository `appsec-core-ontology-research/papers/08-pipeline-primitive-demonstration/artifacts/` (for the three artefact classes whose origin repositories are not publicly accessible: the knowledge graph runtime v1.2, the substrate-grounding evidence + gap-analysis outputs, and the closure brief), the public Manual repository (for the Manual prose corpus), and a cross-cutting figshare bundle DOI assigned at manuscript submission. The ledger anchor's commit pointers are the basis from which the public deposits are mirrored bit-identically.

9.1 Closure ledger snapshot

The cycle's closure-state ledger anchor `cycle-b-frozen-2026-05-12` records four artefact-class closure commits with their corresponding annotated-tag-object SHAs for bit-identical verification at audit:

Artefact class	Origin visibility	Closure commit	Tag object
Manual content	PUBLIC	`455124a1`	`c838ea20a830…`
Knowledge graph runtime + Manual ontology V2	INTERNAL	`dacfaca53640…`	`11369ef6cac7…`
Substrate-grounding evidence + gap-analysis outputs	PRIVATE	`d5da1a0`	`7bd4404088fc…`
Closure brief	PRIVATE	`db60b1b`	`01f2b3e8c65e…`

Table 5. Cycle-closure ledger snapshot at `cycle-b-frozen-2026-05-12`. Each of the four artefact classes is pinned at the closure commit shown in its origin repository (the Manual is independently public at `SbD-ToE/sbd-toe-manual`; the other three origin repositories are not publicly accessible). The per-artefact-class tag object SHA enables bit-identical verification of the closure state against the public deposits of Section 11.5, which mirror these snapshots to publicly citable surfaces. Tag-object SHAs are shown in 12-char prefix form; the full 40-char SHA-1 values are documented in this paper's closure brief manifest (Section 11.5).

9.2 Knowledge graph runtime v1.2 surface

The knowledge graph runtime consumer contract at version 1.2 (published as part of this paper's KG deposit) defines the public consumption surface for the V1 sub-tier. Three Manual-to-ontology linkage record families are surfaced at `data/publish/runtime/v1/`:

Surface	File	Records
Traceability linkage (5-section schema)	`manual_rastreabilidade.jsonl`	1,105
Maturity-progression linkage	`manual_maturity_progression.jsonl`	336
Threat-mitigation linkage	`manual_threat_mitigation.jsonl`	523
Total Manual-to-ontology linkage	(three JSONL files)	1,964

Table 6. Knowledge graph runtime v1.2 Manual-to-ontology linkage surface at the cycle's closure. The traceability linkage records follow the 5-section schema introduced at the cycle's closure (with

manual_v2_anchor, manual_section_anchor, methodology_label, and external-source grounding fields per Section 4); the maturity-progression and threat-mitigation linkages are new at v1.2 (§1.7 and §1.8 of the consumer contract), propagating the §26 methodology vocabulary into the two additional document families introduced at Section 5.6. The KG also publishes V1 entity surfaces (245 entities across control_objectives.json, practices.json, mechanisms.json, artifacts.json, and slices.json) and 529 OWL relation triples at relations.jsonl; these are consumed by the traceability linkage records as foreign-key references. The 245 V1 entity surfaces published at the KG runtime v1.2 are a runtime-bound subset of the 259 typed instances published by [4] (75 Control Objectives + 69 Practices + 58 Mechanisms + 57 Artifacts); the 14-entity delta corresponds to declarative-class entities (EvidencePattern and Signal instances are schema-ready in V1.1 with 0 populated instances at the cycle's closure, per Section 9.4) and to entities outside the runtime substrate-grounding scope as reported by [4].

9.3 Knowledge graph tag predecessor chain

The v1.2 runtime is the cycle-close state of a three-version chain at the V1 sub-tier; all three tags share the common prefix kg-v1-cycle-b- in the knowledge-graph origin repository:

Version	Tag suffix	Date
v1.0	iter-2-aligned	2026-05-11
v1.1	iter-3-aligned	2026-05-11
v1.2	run-2-aligned	2026-05-12

Table 6a. Knowledge graph V1 sub-tier tag chain at the cycle close. Tag suffixes are appended to the common prefix kg-v1-cycle-b- (e.g., the v1.2 full tag is kg-v1-cycle-b-run-2-aligned-2026-05-12).

The substantive content at each version is as follows: v1.0 is the first V1 KG re-compile after the cycle's Manual content extension (Section 5.2), with traceability under the source-first row layout. v1.1 is a re-compile after entity-first traceability re-emission (Section 5.3); 57 Artifact placeholders are surfaced from OWL and the future-work entries register is introduced. v1.2 is the re-compile after Manual ontology V2 vocabulary integration and methodology propagation to maturity and threat families (Sections 5.5–5.6); the 5-section traceability schema is introduced with §26 methodology labels, and the maturity-progression and threat-mitigation linkage families are added. The v1.2 tag is the canonical KG version referenced by the cycle's closure ledger snapshot cycle-b-frozen-2026-05-12 (Section 9.1) and is the version deposited at the public research repository under this paper's deposit chain (Section 11.5).

9.4 Consumer contract v1.2

The consumer contract at v1.2 declares additive backward compatibility with the v1.0 and v1.1 tagged checkouts: consumers operating against the prior tags continue to read the schema they were authored against; v1.2 additions are surfaced on the main branch. The additive-evolution model is consistent with the ontology-evolution discipline distinguished from schema evolution by Noy and Klein [17]: v1.2 additions surface as new linkage families without altering axioms or instances published at v1.0 / v1.1, so prior consumers are not invalidated by the cycle-close consumer-contract evolution. The contract identifies three top-level consumer surfaces: public/base (canonical semantic ontologies and substrate indexes), public/runtime/v1 (the V1 deterministic surface reported in Sections 9.2 and 9.3), and public/serving (MCP and vector retrieval skins derived from the canonical substrate). Consumer guidance prescribes loading Manual ontology V2 first, then AppSec Core V1.1 when normalisation

reasoning is needed; `EvidencePattern` and `Signal` entities are first-class in V2 (213 and 23 instances respectively) and remain schema-ready in V1.1 with 0 populated instances at the cycle's closure.

---

10. Future Work and Limitations

The cycle's execution produces a closed-state artefact pair (Manual + KG joint snapshot) and a transparent registration of items deferred to subsequent cycles or distinct programme streams. This section enumerates the registered items; none of them is a claim of imminent execution, and each is documented at the cycle's closure-state ledger snapshot `cycle-b-frozen-2026-05-12` (Section 9.1) and mirrored to the public deposits of Section 11.5.

10.1 Content-authoring registration

One V1 substantive entity carries the Gap label at the cycle's closure: `ACM-IVF-004`, a centralised error-translation-and-redaction mechanism associated with chapter 06-desenvolvimento-seguro. The cycle did not author Manual prose for this entity, per the anti-rush content discipline of Section 4.4; the entity is registered for a subsequent cycle's authoring scope, under the false-positive caveat carried by the deterministic refinement step (Sections 6.4 and 10.7). Topical overlap with existing Manual content (the `VAL-006/ERR` family in chapter 02 and the LLM input-handling section in chapter 06) is documented in this paper's closure brief (Section 11.5); a future authoring cycle may consolidate the topic into a single chapter-06 section or restructure the relationship to the adjacent `VAL-006/ERR` content, or — under the §10.7 caveat — reclassify the entry as Semantic if subsequent validation surfaces Manual coverage under terminology absent from the cycle's keyword profile.

10.2 Stream 1 — Formal alignment between AppSec Core V1 and Manual ontology V2

The two ontology layers operate in overlay relationship at the cycle's closure (Section 4.3). A formal alignment artefact — for instance, declared SKOS [18] or EDOAL mappings between V1 and V2 entity types, within the broader ontology-matching tradition catalogued by Euzenat and Shvaiko [19] — would substitute the current overlay convention with a citable alignment artefact and enable bidirectional consistency verification. The alignment is registered as a distinct programme stream; it is not within this cycle's execution scope.

10.3 Stream 2 — Formal apparatus for Manual ontology V2

Manual ontology V2 is published at the cycle as a semi-formal YAML vocabulary with type definitions, authority classes, source modes, and confidence models (Section 4.3). A formal apparatus comparable to AppSec Core V1's OWL + SHACL composition (Section 3.2) — an OWL 2 DL export and a SHACL apparatus over Manual ontology V2 — would enable automated corpus validation against the Manual ontology V2 schema. The apparatus is registered as a distinct programme stream.

10.4 V3 verification layer

Manual ontology V2 declares a `verification_scope.status: deferred_to_v3` field at the cycle's closure, registering the verification surface for a successor ontology version. The deferral acknowledges that the current pipeline (Section 3) operates without a programme-wide verification layer beyond the per-cycle joint ratification of Section 9; a V3 ontology iteration would author the verification surface as a first-class artefact.

10.5 Pipeline re-execution at subsequent cycles

The pipeline reported in Section 3 is positioned for re-execution at subsequent cycles. A subsequent cycle may operate over an expanded external-source corpus, an evolved V1 ontology version, or an updated Manual content state. The cycle's pipeline architecture and closure-mechanism vocabulary are designed to operate against the same surface structure at substantively different substrate compositions; whether the empirical mix reported in Sections 6 and 8 generalises is an empirical question for those subsequent cycles, not foreclosed by the pipeline's design.

10.6 Boundaries of the cycle's claims

The cycle's empirical claims are bounded by three scoping conditions stated throughout the paper and consolidated here: (i) the substrate at the cycle is the 31-source corpus normalised through the normalized substrate; (ii) the V1 ontology is the v1.1-fair-baseline state with 259 typed instances across 10 slices; (iii) the Manual is the practitioner-authored corpus at the cycle's closure state across 15 chapters and four document families. Claims about pipeline behaviour at substantially different substrate compositions are out of scope. Claims about the substantive correctness of the V1 ontology or its normalisation of source frameworks are out of scope per Section 1.4: the pipeline measures grounding-against-corpus through `M_sbd`-analogous closure mechanisms, not the accuracy of the underlying ontology or its source-framework normalisation. Claims about Manual editorial correctness (the substantive engineering content of the Manual prose) are out of scope: the pipeline measures the routing of Manual prose against the V1 substrate-grounded overlay, not the prose's engineering correctness.

10.7 Independent validation of the deterministic keyword refinement

The §26 keyword-based refinement step (Section 4.1; Semantic requires $\geq$3 distinct keywords with $\geq$5 total occurrences in the V1-slice-expected chapter; Partial requires the same evidence threshold met only in chapters other than the slice anchor; Gap requires the threshold met nowhere) operates deterministically against the chapter prose corpus, so its outputs are reproducible from the cycle's closure state (the keyword index, chapter prose, and per-entity output are jointly recoverable from this paper's deposit chain, Section 11.5). The refinement does not, however, carry an independent validation surface: no inter-rater agreement (Cohen's κ or analogous) is reported, no ground-truth panel external to the deterministic keyword detection has verified per-entity label assignments, and the keyword vocabularies driving the threshold counts have not been benchmarked against a hand-coded reference. The §6.5 finding that 37 of 38 detection-candidate gaps are reclassified as non-content deficits is therefore reported as the deterministic output of the refinement step, not as independently validated truth; downstream consumers should read the §26 distribution as a substrate-derived approximation that subsequent cycles, or an external validation study registered separately, may calibrate against an independent reading.

---

11. Reproducibility

11.1 Citation convention

Artefact references in this paper follow one of three forms:

- Cross-paper citations (`[4]`, `[5]`, etc.) for artefacts that are deliverables of other programme papers; the cited paper carries its own deposit chain. The AppSec Core V1 ontology, the SHACL apparatus, and the embeddings v1.1 release are deliverables of [4]; the normalised substrate produced by the claim-centric two-stage pipeline is a deliverable of [5]. References [4] and [5] are forthcoming programme papers; at the time of this paper's deposit they are canonical via the con-

struction tags recorded in the References section (`v2.0.0-construction-p6-final-draft` and `v2.0.0-construction-p7-final-draft`), with OSF deposits pending. The deposit sequence preceding this paper's submission resolves the construction tags to citable OSF preprints; the citation forms in this paper are read against the construction tags until that resolution and against the OSF DOIs thereafter.

- Public deposit paths for this paper's outputs. The deposits are distributed across two public surfaces because only one of the four artefact-class origin repositories is publicly accessible:
  - the public research repository `appsec-core-ontology-research/papers/08-pipeline-primitive-demonstration/artifacts/` receives the three artefact classes whose origin repositories are not publicly accessible (the knowledge graph runtime v1.2 + Manual ontology V2 YAML, the substrate-grounding evidence + gap-analysis outputs, and the closure brief);
  - the public Manual repository `SbD-ToE/sbd-toe-manual` at the closure tag `cycle-b-frozen-2026-05-12` is the canonical citation form for the Manual prose corpus (the only artefact class whose origin repository is independently public).
- figshare bundle DOI — a cross-cutting cycle-bundle deposit at figshare (analogous to the programme's precedent `cycle-a-frozen-2026-05-08` bundle) is assigned a DOI at manuscript submission and is the canonical archival citation form thereafter.
- Closure ledger anchor `cycle-b-frozen-2026-05-12` (Section 9.1) is the programme's internal-governance ledger snapshot pinning the closure commits across the four artefact-class origin repositories. It is the basis from which the public deposits above are mirrored, and is referenced in this paper for bit-identical audit; it is not the external citation form, and external readers cite the public deposit paths (or the figshare DOI once assigned).
- Internal SHA anchors (e.g., SUPPLIER SHA-256 `596783ed…` for the substrate) are recorded for bit-identical reproducibility verification independent of the deposit hosting.

11.2 Pipeline-stage citation chain

Each pipeline stage of Section 3 is reproducible from artefacts published under either a cross-cited paper's deposit chain or this paper's deposit chain (Section 11.5):

- Stage 1 (external-source corpus): the 31-source corpus is the cycle-close substrate input established by [5]; per-source provenance (retrieval receipts with origin URL and SHA-256 hashes) is published as part of [5]'s deposit chain.
- Stage 2 (normalization): the claim-centric two-stage normalization pipeline (lifting followed by SBERT-based similarity grounding) is documented in [5]; design rationale and per-stage scripts are published as part of [5]'s deposit chain.
- Stage 3 (normalized substrate): the SUPPLIER artefact carries SHA-256 `596783ed984d…` (full 64-char digest in [5]'s bundle manifest) and is reproducible via the per-source pipeline scripts published as part of [5]'s deposit chain.
- Stage 4 (V1 ontology binding): the AppSec Core V1 ontology — canonical OWL/Turtle export (SHA-256 `588598ff…`), entity inventory YAML index (SHA-256 `cdc440e8…`), and the SHACL apparatus with schema-derived shapes (SHA-256 `d30e716f…`) and consumer-conformance shapes (SHA-256 `0b782136…`) — is published by [4] under that paper's deposit chain.
- Stage 5 (coverage analysis): the per-entity source map, the gap-classification outputs, and the generator scripts for the deterministic coverage detection and keyword-based reclassification are published as part of this paper's deposit chain (Section 11.5).
- Stage 6 (Manual content surfaces and KG): the Manual content, the knowledge graph runtime

v1.2 (with its consumer contract specification), and the Manual ontology V2 YAML are published as part of this paper's deposit chain (Section 11.5).

A complete per-artefact SHA-256 inventory accompanying the deposit chain is documented in this paper's closure brief (Section 11.5) and enables bit-identical reproducibility verification across the deposit DOIs.

11.3 Build environment for bit-identical reproducibility

The embeddings v1.1 NPZ output (and any downstream computation that depends on it) is bit-identical reproducible under a pinned eleven-dimension build environment: Darwin x86_64 host architecture; Python 3.10.1; `transformers` 4.57.1; `torch` 2.2.2; `numpy` 1.24.4; `pyshacl` 0.31.0; `rdflib` 7.6.0; the Sentence-BERT model [20] `sentence-transformers/all-MiniLM-L6-v2`; HuggingFace model revision SHA `c9745ed1d9f2…`; encoder maximum tokens 256; attention-mask-weighted mean pooling; L2 normalisation (p=2, dim=1). Identity match across all eleven dimensions is required for bit-identical NPZ output. Cross-architecture identity is not guaranteed; the embeddings manifest at the embeddings manifest published by [4] records `platform.machine` for diagnostic purposes.

11.4 OWL and SHACL validation evidence

The V1 ontology at the v1.1-fair-baseline tag — "FAIR" denoting the FAIR Guiding Principles for scientific data management and stewardship as articulated by Wilkinson et al. [21] — passes external-tool validation as reported in [4]: the OOPS! ontology pitfall scanner [11] reports zero Critical and zero Important pitfalls plus two Minor pitfalls documented as design choices; the FOOPS! FAIR ontology pitfall scanner [12] (which evaluates conformance with the Wilkinson et al. FAIR principles [21]) reports 13 of 15 binary checks passed, with the two remaining gaps (persistent-URL registry and DOI identifier) gated on external prerequisites scheduled for post-publication resolution. SHACL validation under both `pyshacl 0.31.0` (the W3C-canonical reference implementation) and the in-house bounded validator reports `conforms=True` with zero violations across the composed apparatus (six schema-derived NodeShapes plus five hand-maintained consumer-conformance Claim shapes; 396 shape triples total; 1,970 data triples).

11.5 Public deposit chain

The cycle's Manual + knowledge-graph joint snapshot is the citable deliverable referenced by this paper's reproducibility chain. The snapshot encompasses four artefact classes. All four origin repositories sit under the `SbD-ToE/` GitHub organisation, and public deposit paths (other than the Manual's own public origin) are written relative to the prefix `appsec-core-ontology-research/papers/08-pipeline-primitive-demonstration/` — both shorthands are defined once here and reused below. Only one of the four origin repositories (the public Manual repository `SbD-ToE/sbd-toe-manual`) is publicly accessible, so the remaining three artefact classes are deposited at the public research repository as part of this paper's published-paper folder. The on-disk public-deposit layout uses six subtrees totalling 28 entries. A cross-cutting figshare bundle deposit (analogous to the programme's precedent `cycle-a-frozen-2026-05-08` bundle) is assigned a DOI at manuscript submission and becomes the canonical archival citation form thereafter.

The four artefact classes are enumerated below; each entry records the on-disk content, the origin repository (relative to the `SbD-ToE/` organisation) and its visibility, the public deposit surface, the closure commit at the origin repository, and the archival DOI status:

- Manual prose corpus — 15 chapters across 4 document families, 322 markdown files.
  - Origin: `sbd-toe-manual` (PUBLIC).

  - Public deposit: cited at its own public origin repository at the Git tag `sbd-toe-manual@cycle-b-frozen-2026-05-12`.
  - Closure commit at origin: `455124a1`.
  - Archival DOI: figshare DOI assigned at submission.
- Knowledge graph runtime v1.2 + Manual ontology V2 — 1,964 Manual-to-ontology linkage records, 245 V1 entity surfaces, 529 OWL relation triples, and the Manual ontology V2 YAML, organised into three sub-subtrees: `kg_v1_2/` (11 entries: traceability, maturity, and threat-linkage tables, V1 entity tables, v1 manifest), `kg_indexes/` (6 entries: chunks-layer bundle-complete extension), and `manual_freeze/` (1 entry: KG-canonical Manual freeze ref).
  - Origin: `sbd-toe-knowledge-graph` (INTERNAL).
  - Public deposit: `artifacts/kg_v1_2/`, `artifacts/kg_indexes/`, `artifacts/manual_freeze/`.
  - Closure commit at origin: `dacfaca53640`….
  - Archival DOI: figshare DOI assigned at submission.
- Substrate-grounding evidence + gap-analysis outputs — 8 entries: per-source retrieval receipts, substrate SUPPLIER artefact, gap-classification outputs and the `phase2_3/` subdirectory, per-entity source map.
  - Origin: `external-sources-inventory` (PRIVATE).
  - Public deposit: `artifacts/gap_analysis/`.
  - Closure commit at origin: `d5da1a0`.
  - Archival DOI: figshare DOI assigned at submission.
- Closure brief — 1 entry (per-artefact content inventory, per-artefact SHA-256 entries, ledger tag-object SHA inventory) plus 1 helper-script entry under `scripts/`.
  - Origin: `DevelopmentGovernance` (PRIVATE).
  - Public deposit: `artifacts/closure_brief/`, `artifacts/scripts/`.
  - Closure commit at origin: `db60b1b`.
  - Archival DOI: figshare DOI assigned at submission.

The Manual is the only artefact class whose origin repository is independently public, and is therefore cited at its origin repository under the closure Git tag; the remaining three artefact classes are deposited at the public research repository under this paper's published-paper folder because their origin repositories are not publicly accessible. The closure commits and the cycle's closure ledger snapshot `cycle-b-frozen-2026-05-12` (Section 9.1) enable bit-identical verification of the public deposits against the origin closure state. The Manual freeze ref artefact deposited at `artifacts/manual_freeze/manual_freeze_ref.json` (KG-canonical, Codex-authored at the origin path `data/publish/runtime/v1/manual_freeze_ref.json`) is pinned at the KG programme tag `kg-v1-cycle-b-manual-ref-2026-05-14`, distinct from the multi-repo `cycle-b-frozen-2026-05-12` closure ledger anchor; this artefact provides a stable locator for the independently public Manual prose corpus at its closure-tag deposit at `SbD-ToE/sbd-toe-manual`. The cross-cutting figshare bundle DOI, when assigned at submission, becomes the canonical archival citation form for the snapshot.

The public deposits are mandatory prerequisites for the paper's reproducibility chain: external reviewers and future consumers reference the public deposit paths under this paper's `artifacts/` subtree (relative to the prefix declared above) and the Manual repository at `sbd-toe-manual@cycle-b-frozen-2026-05-12` until figshare DOI assignment, and the figshare DOI thereafter. The closure ledger anchor `cycle-b-frozen-2026-05-12` (Section 9.1) is the programme-governance snapshot from which the public deposits are mirrored; it is not the external citation form. The V1 ontology archive referenced throughout Sections 3, 4, and 11.2 is the deliverable of the artefact paper [4] and is deposited

under that paper's deposit chain at the public research repository under `papers/06-appsec-core-v1-formalized-artefact/artifacts/` (per the programme's per-paper deposit convention); the substrate produced by the normalization pipeline of [5] is deposited under that paper's deposit chain at `papers/07-method-dsr-cycle/artifacts/`.

References

[1] P. Farinha. AppSec Core: A Normalised Ontology for Security Requirements Across Heterogeneous Frameworks. Open Science Framework Preprint, 2026. doi:10.17605/OSF.IO/WG8PV.

[2] P. Farinha. Coverage-Preserving Knowledge Compilation. Open Science Framework Preprint, 2026. doi:10.17605/OSF.IO/A6ZFJ.

[3] P. Farinha. *Research Programme Prospectus*. Independent research preprint, 2026. doi:10.17605/OSF.IO/7T849.

[4] P. Farinha. AppSec Core v1: A Formalized Normalization Ontology for Application Security. Independent research preprint, 2026. doi:10.17605/OSF.IO/U9CRD. Canonical via construction tag `v2.0.0-construction-p6-final-draft`.

[5] P. Farinha. Pressure-Testing AppSec Core: A Design Science Cycle for Bounded-Ontology Evolution Under Heterogeneous Application-Security Sources. Independent research preprint, 2026. doi:10.17605/OSF.IO/3E8G5. Canonical via construction tag `v2.0.0-construction-p7-final-draft`.

[6] A. R. Hevner, S. T. March, J. Park, and S. Ram. "Design Science in Information Systems Research". *MIS Quarterly*, 28(1):75–105, 2004. doi:10.2307/25148625.

[7] R. J. Wieringa. *Design Science Methodology for Information Systems and Software Engineering*. Springer, 2014. doi:10.1007/978-3-662-43839-8.

[8] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. "A Design Science Research Methodology for Information Systems Research". *Journal of Management Information Systems*, 24(3):45–77, 2007. doi:10.2753/MIS0742-1222240302.

[9] P. Farinha. Empirical Research Design: A Security-by-Design Evaluation Framework for Ontology-Grounded Code Generation. Open Science Framework Pre-Registration, 2026. doi:10.17605/OSF.IO/H5AJE.

[10] P. Farinha. MCP Dual-Mode Instrument Specification for Ontology-Grounded Secure Code Generation. Open Science Framework Component Preprint, 2026. doi:10.17605/OSF.IO/KH8Y7.

[11] M. Poveda-Villalón, A. Gómez-Pérez, and M. C. Suárez-Figueroa. "OOPS! (OntOlogy Pitfall Scanner!): An On-line Tool for Ontology Evaluation". *International Journal on Semantic Web and Information Systems*, 10(2):7–34, 2014. doi:10.4018/ijswis.2014040102. Tool URL: https://oops.linkeddata.es/.

[12] D. Garijo, O. Corcho, and M. Poveda-Villalón. "FOOPS!: An ontology pitfall scanner for the FAIR principles". In *Proceedings of the ISWC 2021 Posters, Demos and Industry Tracks*, CEUR Workshop Proceedings Vol. 2980, 2021. URL: https://ceur-ws.org/Vol-2980/. Tool URL: https://foops.linkeddata.es/.

[13] N. F. Noy. "Semantic integration: a survey of ontology-based approaches". *ACM SIGMOD Record*, 33(4):65–70, 2004. doi:10.1145/1041410.1041421.

[14] P. Cimiano. *Ontology Learning and Population from Text: Algorithms, Evaluation and Applications*. Springer, 2006. doi:10.1007/978-0-387-39252-3.

[15] C. D. Manning, P. Raghavan, and H. Schütze. *Introduction to Information Retrieval*. Cambridge

University Press, 2008. Open-access edition: https://nlp.stanford.edu/IR-book/.

[16] M. K. Buckland and F. C. Gey. "The relationship between recall and precision". *Journal of the American Society for Information Science*, 45(1):12–19, 1994. doi:10.1002/(SICI)1097-4571(199401)45:1<12::AID-ASI2>3.0.CO;2-L.

[17] N. F. Noy and M. Klein. "Ontology evolution: Not the same as schema evolution". *Knowledge and Information Systems*, 6(4):428–440, 2004. doi:10.1007/s10115-003-0137-2.

[18] A. Isaac and E. Summers. *SKOS Simple Knowledge Organization System Primer*. W3C Working Group Note, 18 August 2009. URL: https://www.w3.org/TR/skos-primer/.

[19] J. Euzenat and P. Shvaiko. *Ontology Matching*. Springer, 2nd edition, 2013. doi:10.1007/978-3-642-38721-0.

[20] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019. doi:10.18653/v1/D19-1410. arXiv:1908.10084.

[21] M. D. Wilkinson, M. Dumontier, IJ. J. Aalbersberg, et al. "The FAIR Guiding Principles for scientific data management and stewardship". *Scientific Data*, 3:160018, 2016. doi:10.1038/sdata.2016.18.

AI Use Statement

Generative-language-model tools were used by the author during the preparation of this paper for drafting and editing of prose, under the author's review at every step. All scientific claims, methodological decisions, evidence interpretation, and the empirical attribution of the pipeline's outputs at the cycle's closure (§§5–8) are the author's responsibility. The pipeline this paper publishes — its composition of prior programme artefacts and its operational closure mechanisms — was developed and exercised through the design science cycle reported in [5] and the operational instantiation reported in §5; no generative language model is invoked in the pipeline's runtime execution beyond the deterministic Sentence-BERT [20] sentence encoder used by the substrate-grounding stage (a deterministic encoder, not a generative model). The cycle's frozen ratification at the closure ledger anchor `cycle-b-frozen-2026-05-12` was executed by programme governance under the author's authorisation.